Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

## Appendix A

# Basics of the C programming language

The following is a brief overview of the features of C used in CSparse, including a description of how each feature translates into MATLAB. See [141] for a full yet concise description of the C programming language.

## Variables

Six of C's basic variable types are used in CSparse:

| | |
|---|---|
| `int` | an integer |
| `unsigned int` | an integer that is always positive |
| `double` | a double-precision floating-point value |
| `size_t` | an integer large enough to hold a pointer |
| `char` | a character |
| `void` | an object of no specific type used for pointers |

In MATLAB, variables do not need to be declared (except with the rarely used `global` statement). They must be declared in C. For example, the following C statements declare integer scalars `i` and `j` and a double-precision scalar `x`. Declarations can include initializations as well. C also includes a `complex` type.

```
int i ;
double x ;
int j = 0 ;
```

In C, an array is represented by a *pointer* to an address in memory containing the first element of the array. The following is also an example of a `/* comment */` in C.

```
int *s ;            /* declares a pointer to an int */
double *x ;         /* declares a pointer to a double */
void *p ;           /* declares a pointer to "void" */
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

## Statements and Operators

Logical operators are slightly different in C and MATLAB. In C, `!` is the logical negation operator, which is the same as the tilde operator (`~`) in MATLAB. The not-equals operator is `!=` in C and `~=` in MATLAB. In both languages, logical *or* (`||`) and logical *and* (`&&`) are the same. The caret (`^`) operator in C is the exclusive-or (`bitxor` in MATLAB). Logical statements are evaluated left to right; evaluation terminates as soon as the result is known.

A pointer whose value is `NULL` is special (`NULL` is zero). It points to nothing. In both C and MATLAB, true is nonzero and false is zero. Thus if `x` is a pointer, the expression `!x` is true if `x` is a `NULL` pointer and false otherwise.

In C, single statements must always be terminated with a semicolon. A compound statement can be used in C wherever a single statement can be used; it starts with `{` and ends with `}`. The latter is analogous to `end` in MATLAB. Assignments can be used as expressions in C but not in MATLAB. For example, `x=y=0 ;` in C sets both `x` and `y` to zero. The comma operator can be used to separate a pair of expressions. Both expressions are evaluated, and the result is the value of the right operand. It is used only in `for` loops in CSparse. C does not have matrix operators. The ternary operator `?:` is unique to C. In C, the statement

```
x = (y > 0) ? z : w ;
```

is equivalent to the MATLAB statements

```
if (y > 0)
    x = z ;
else
    x = w ;
end
```

If `p` is a pointer, `*p` denotes the value stored at that memory location; that is, the unary `*` operator *dereferences* a C pointer to obtain the value stored at the memory location referred to by the pointer. The expression `&x` is a pointer to the variable `x`. Arithmetic can be performed on C pointers. If `x` is a pointer to an array of size 10, `x[i]` is the `i`th entry in the array, where `i` can be in the range `0` to `9`. In MATLAB, the array is `x(1)` through `x(10)`. Pointer arithmetic is always done in units of the size of the type of variable pointed to by the pointer. Thus, `*(x+i)` is identical to `x[i]`. CSparse uses very little pointer arithmetic of the form `x+i`, except when laying out subarrays from a larger workspace. It uses the form `x[i]` extensively.

The `++` and `--` operators are unique to C. As stand-alone statements, `x++` and `++x` are both the same as `x = x + 1`. When used in expression, `x++` increments `x` after its value is used in the expression, and `++x` increments `x` before it is used. Thus,

```
y = x++ ;
```

is the same (in both C and MATLAB) as

```
y = x ;
x = x + 1 ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

and

```
y = ++x ;
```

is the same (in both C and MATLAB) as

```
x = x + 1 ;
y = x ;
```

Likewise, the C statement

```
x [k++] = i ;
```

is the same as the C statements

```
x [k] = i ;
k = k + 1 ;
```

C has a suite of assignment operators that modify the left-hand side. The following table shows five of these and their equivalents using the regular assignment in C.

| | |
|---|---|
| `x += 2` | `x = x + 2` |
| `x -= 2` | `x = x - 2` |
| `x /= 2` | `x = x / 2` |
| `x *= 2` | `x = x * 2` |
| `x %= 2` | `x = x % 2` |

The `%` operator in C is the `rem` function in MATLAB, except that the meaning of `a % b` when either `a` or `b` are negative is machine dependent (it is used in CSparse only for positive numbers).

Variables can be *typecast* into values of a different type. For example, to convert an `int` to a `double`,

```
x = (double) i ;
```

This conversion is done automatically by the assignment `x=i` and when variables of one type are passed to a function expecting another type, so it is rarely needed.

## Control structures

The `while` loop is almost the same in C and MATLAB. These two code fragments are the same, the first in C and the second in MATLAB:

```
while (x < 10)
{
    x = x + i ;
}
```

```
while (x < 10)
    x = x + i ;
end
```

Both C and MATLAB have a `for` loop, but they differ in how they work. These two code fragments are the same in C and MATLAB, respectively.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
for (i = 0 ; i < n ; i++)
{
    x = x + i ;
}

for i = 0:n-1
    x = x + i ;
end
```

The C `for` loop has four components. In general,

```
for ( initialization ; condition ; post )
{
    body
}
```

is identical to

```
initialization
while ( condition )
{
    body
    post
}
```

Thus, these two loops are identical in C:

```
for (i = 0 ; i < n ; i++)
{
    x = x + i ;
}

i = 0 ;
while (i < n)
{
    x = x + i ;
    i = i + 1 ;
}
```

Any of the four components of a `for` loop can be empty. Thus `for (;;) ;` is an infinite loop. The `continue` and `break` statements are identical in C and MATLAB; the former causes the next iteration of the nearest enclosing `for` or `while` loop to begin, the latter terminates the nearest enclosing loop immediately.

## Functions

Parameters to C and MATLAB functions are both passed by value; a C function and a MATLAB M-file function cannot modify their input parameters. However, a pointer can be passed by value to a C function, and the *contents* of what that pointer points to can be modified. Most CSparse functions do not modify their inputs. The `const` keyword when used in a function header or prototype declares that the function does not modify the contents of an array (or, equivalently, what a pointer argument points to).

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

A C function can return only a single value to its caller. This value can be a pointer. For example, the `cs_malloc(n,b)` function returns a pointer to a newly allocated (but uninitialized) memory space of size large enough to hold an array of `n` items each of `b` bytes. The `sizeof` operator is applied to a C type and returns the number of bytes in an object of that type (normally, `sizeof(int)` is four, and `sizeof(double)` is eight). The C `return` statement is like the MATLAB `return`, except that it also defines the value returned to the caller. The following C function and MATLAB function are the same, except MATLAB always uses double-precision floating-point values to represent its integers (called a *flint* in MATLAB).

```
int *myfunc (int n)
{
    int *p, i ;
    p = cs_malloc (n, sizeof (int)) ;
    for (i = 0 ; i < n ; i++) p [i] = i ;
    return (p) ;
}
```

In MATLAB, the same function is

```
function p = myfunc (n)
p = 0:n-1 ;
```

These functions are called in the same way in C and MATLAB: `p=myfunc(n);`. C functions that are private to a specific source code file are declared `static`. They can be called only by other functions in that same file, just like a nested or private function in MATLAB. A *prototype* is a statement declaring the name, parameters (and their type), and return value of a function. These are normally placed in an *include file* (described below), so that they can be incorporated into any code that calls the function. Prototypes for functions returning an `int` are not strictly required. However, if the declaration and use of the function are different and no prototype is present, the results are unpredictable. Prototypes should always be used in well-written C code. MATLAB has no prototypes but checks each usage of a function as it is called at run time. The prototype for `myfunc` is

```
int *myfunc (int n) ;
```

Both C and MATLAB can work with pointers to functions (called function handles in MATLAB). Consider the `cs_fkeep` function with prototype:

```
int cs_fkeep (cs *A, int (*fkeep) (int, int, double, void *), void *other) ;
```

Its second argument is a pointer to a function with four parameters (two `int`'s, a `double`, and a pointer to `void`). The function `cs_fkeep` calls the `fkeep` function for each entry in the matrix. An example of the use of `cs_fkeep` is in the `cs_droptol` function. Note that `cs_droptol` passes a pointer to `cs_tol` to `cs_fkeep`.

```
static int cs_tol (int i, int j, double aij, void *tol)
{
    return (fabs (aij) > *((double *) tol)) ;
}
int cs_droptol (cs *A, double tol)
{
    return (cs_fkeep (A, &cs_tol, &tol)) ;    /* keep all large entries */
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

## Data structures

Both C and MATLAB can create a compound object called a `struct`. In C, these must be declared statically. They can be dynamically created in MATLAB. The following code fragments are identical. In C, the `mystuff` type is first defined as a structure containing a scalar integer and a pointer to a double. It is then used in a declaration statement to define `f` of type `mystuff`.

```
typedef struct mystuffstruct
{
    int i ;
    double *x ;
} mystuff ;
mystuff f ;
f.i = 3 ;
f.x = cs_calloc (4, sizeof (double)) ;
```

In MATLAB, the `struct` function may be used, or the components of a structure may be defined one at a time:

```
g = struct ('i', 3, 'x', zeros (4,1)) ;
f.i = 3 ;
f.x = zeros (4,1) ;
```

In the C example above, `f` is declared as an object of type `mystuff`. A pointer to an object of this type can also be defined. Accessing the contents of a `struct` with a pointer is different, using the `->` operator. The fragment below is identical to the C code above, except that it uses a pointer instead. The `->` operator is used exclusively in CSparse, instead of the dot (`.`). In C, if `p` is a pointer to a `struct` containing the member `i`, the expressions `p->i` and `(*p).i` are identical.

```
typedef struct mystuffstruct
{
    int i ;
    double *x ;
} mystuff ;
mystuff *p ;
p = cs_malloc (1, sizeof (mystuff)) ;
p->i = 3 ;
p->x = cs_calloc (4, sizeof (double)) ;
```

## Examples

Consider the following statement from `cs_pvec`:

```
for (k = 0 ; k < n ; k++) x [k] = b [p ? p [k] : k] ;
```

A more leisurely way of expressing this is

```
for (k = 0 ; k < n ; k++)
{
    if (p != NULL)
    {
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
            x [k] = b [p [k]] ;
        }
        else
        {
            x [k] = b [k] ;
        }
    }
```

Both examples shown above compile into code that is equally fast. The former is just more concise. The statements from `cs_transpose`

```
Ci [q = w [Ai [p]]++] = j ; /* place A(i,j) as entry C(j,i) */
if (Cx) Cx [q] = Ax [p] ;
```

are a more concise form of the following:

```
i = Ai [p] ;
q = w [i] ;
w [i] = w [i] + 1 ;
Ci [q] = j ;
if (Cx != NULL)
{
    Cx [q] = Ax [p] ;
}
```

The left-to-right evaluation of logical statements in C is exploited by this example from `cs_multiply`.

```
if (nz + m > C->nzmax && !cs_sprealloc (C, 2*(C->nzmax)+m))
{
    return (cs_done (C, w, x, 0)) ;             /* out of memory */
}
```

If `nz+m` is less than the space available in `C` (`C->nzmax`), the first operand of `&&` is false, and the second expression is not evaluated. Thus, `cs_sprealloc` is called only if `nz+m > C->nzmax`. If this first expression is true but `cs_sprealloc` returns false, then it failed to allocate sufficient memory. In this case, `cs_done` frees all workspace and the result `C` and returns `NULL` to signify that it ran out of memory.

## C library functions

The following C library functions are used in CSparse:

| | |
|---|---|
| `fabs(x)` | absolute value of `x` |
| `sqrt(x)` | square root of `x` |
| `malloc(n)` | allocates a block of `n` bytes of memory |
| `calloc(n,b)` | allocates a block of `n` items each of size `b` and sets it to zero |
| `free(p)` | frees a block of memory |
| `realloc(p,n)` | changes the size of a block of memory to `n` bytes |
| `printf` | just like `fprintf` in MATLAB |
| `fscanf` | just like `fscanf` in MATLAB |
| `clock` | returns CPU time used (used only in the CSparse demos) |
| `qsort` | sorts an array |
| `rand` | random number generator |
| `srand` | set random number seed |

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

## C preprocessor

The C preprocessor is a text-only preprocessing step, applied to a program before it is compiled. Preprocessor statements start with `#`, usually in the first column. The statements used in CSparse are listed below.

| | |
|---|---|
| `#include` | includes a file |
| `#define` | defines a macro or token |
| `#ifdef` | true if the token is defined |
| `#ifndef` | true if the token is not defined |
| `#else` | the "else" part to an `ifdef` or `ifndef` |
| `#endif` | the "endif" part to an `ifdef` or `ifndef` |

The `#include` statement has one of the forms

```
#include <file.h>
#include "file.h"
```

The only difference between the two is where the C compiler looks for the file called `file.h`. The first one looks in a sequence of predetermined locations (dependent on your compiler and operating system). The second (`"file.h"`) looks first in the same place as the current source file and, failing that, looks in the same place as the `<file.h>` form. This file is copied into the source code that has the `#include` statement before it is compiled.

The `#define` statement can define a token or a macro. A token is a single word that has no parameters. For example, the word `NULL` is defined as `0`, or `((void *) 0)`, in the `<stdio.h>` or `<stdlib.h>` file. The `#define` statement is used in CSparse to define the memory management routines CSparse should use. If CSparse is being compiled in a MATLAB mexFunction, the token `MATLAB_MEX_FILE` is defined, and the MATLAB memory management routines are used (`mxMalloc`, `mxCalloc`, `mxFree`, and `mxRealloc`) instead of their standard C counterparts (`malloc`, `calloc`, `free`, and `realloc`). Consider three macros that are defined in the `cs.h` file:

```
#define CS_MAX(a,b) (((a) > (b)) ? (a) : (b))
#define CS_MIN(a,b) (((a) < (b)) ? (a) : (b))
#define CS_FLIP(i) (-(i)-2)
```

`CS_MAX` and `CS_MIN` are easier-to-read versions of the ternary `?:` operator and compute the maximum and minimum of `a` and `b`, respectively. The `CS_FLIP(i)` macro computes the simple function `-(i)-2`, so named because it "flips" an integer about the pivotal integer `-1`, somewhat analogous to flipping the sign-bit of a number. More precisely, `CS_FLIP(-1)` is `-1`, and for all integers (ignoring overflow) `CS_FLIP(CS_FLIP(i))` equals `i`.